Computational and storage limitations. There is great interest in using mobile devices to control home environmental and security services. When MobileApps are allowed to interact with devices and services in their environment, it is easy to overwhelm the mobile device (storage, processing speed, power consumed) with the sheer volume of information [Spa11]. Developers may need to look for programming shortcuts and means of reducing the demands made on processor and memory resources.

Applications that depend on external services. Building thin mobile clients suggests the need to rely on Web service providers and cloud storage facilities. This increases concerns for both data or service accessibility and security [Rot02].

Testing complexity. Mobile products that run entirely on the device can be tested using traditional software testing methods (Chapters 19 and 20) or using emulators running on personal computers. Thin-client MobileApps are particularly challenging to test. They exhibit many of the same testing challenges found in WebApps, but they have the additional concerns associated with transmission of data through Internet gateways and telephone networks [Was10]. Testing of mobile software products will be discussed in Chapter 21.

13.2 Mobile Development Life Cycle

Burns [Bur16] and her Microsoft colleagues describe a recommendation for an iterative mobile SDLC that contains five major stages:

Inception. Goals, features, and functions of the mobile product are identified to determine the scope and the size of the first increment or feasibility prototype. Developers and stakeholders must be conscious of human, social, cultural, and organizational activities that may reveal hidden aspects of the users' needs and affect the business targets and functionality of the proposed mobile product.

Design. The design includes architectural design, navigation design, interface design, content design. Developers define the app user experience using screen mockups and paper prototypes to help create a proper user interface design that will take different screen sizes and capabilities into account as well as the capabilities of each targeted platform.

Development. Mobile software is coded, functional and nonfunctional. Test cases are created and executed, and usability and accessibility evaluations are conducted as the product evolves.

Stabilization. Most mobile products go through a series of prototypes: feasibility prototype, intended as a proof of concept with perhaps only one complete logic path through the application; alpha prototype, which contains the functionality for minimum viable product; beta prototype, which is largely complete and contains most tested functionality; and lastly the release candidate, which contains all required functionality, for which all scheduled tests have been completed, and which is ready for review by the product owner.

Deployment. Once stabilized, a mobile product is reviewed by a commercial app store and made available for sale and download. For apps intended for internal company use only, a product owner review may be all that is required before deployment.

Mobile development makes use of an agile, spiral engineering process model. The stages are not completed in order like they would be if mobile development was done using the waterfall model. The stages described above are visited repeatedly as developers and stakeholders gain better understanding of the user needs and product business goals.

SAFEHOME



Formulating Mobile Device Requirements

The scene: A meeting room. The first meeting to identify requirements for a mobile version of the *SafeHome* WebApp.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator (pointing at whiteboard): So that's the current list of objects and services for the home security function present in the WebApp.

Vinod (interrupting): My understanding is that people want *SafeHome* functionality to be accessible from mobile devices as well . . . including the home security function?

Marketing person: Yes, that's right . . . we'll have to add that functionality and try to make it context aware to help personalize the user experience.

Facilitator: Context aware in what sense?

Marketing person: People might want to use a smartphone instead of the control panel and avoid logging on to a website when they are in the driveway at home. Or they might not want all family members to have access to the master control dashboard for the system from their phones.

Facilitator: Do you have specific mobile devices in mind?

Marketing person: Well, all smartphones would be nice. We will have a Web version done, so won't the MobileApp run on all of them?

Jamie: Not quite. If we took a mobile phone browser approach, we might be able to reuse a lot of our WebApp functionality. But remember, smartphone screen sizes vary, and they may or may not all have the same touch capabilities. So, at the very least we would have to create a mobile website that takes the features of each device into account.

Ed: Perhaps we should build the mobile version of the website first.

Marketing person: OK, but a mobile website solution wasn't what we had in mind.

Vinod: Each mobile platform seems to have its own unique development environment, too.

Production rep: Can we restrict MobileApp development to only one or two types of smartphones?

Marketing person: I think that might work. Unless I'm mistaken, the smartphone market is dominated by two smartphone platforms right now.

Jamie: There's also security to worry about. We better make sure an outsider can't hack into the system, disarm it, and rob the place or worse. Also, a phone could get lost or stolen more easily than a laptop.

Doug: Very true.

Marketing: But we still need the same level of security . . . just also be sure to stop an outsider from getting in with a stolen phone.

Ed: That's easier said than done and . . .

Facilitator (interrupting): Let's not worry about those details yet.

(Doug, serving as the recorder for the meeting, makes an appropriate note.)

Facilitator: As a starting point, can we identify which elements of WebApp security function are needed in the MobileApp and which will need to be newly created? Then we can decide how many mobile platforms we can support and when we can move forward on this project.

(The group spends the next 20 minutes refining and expanding the details of the home security function.)

13.2.1 User Interface Design

Mobile device users expect that minimal learning time will be required to master a MobileApp. To achieve this, MobileApp designers use consistent icon representations and placement across multiple platforms. In addition, designers must be sensitive to the user's expectation of privacy with regard to the display of personal information on the screen of the mobile device. Touch and gesture interfaces, along with sophisticated voice input and facial recognition, are maturing rapidly [Shu12] and have already become part of the user interface designer's toolbox.

Legal and ethical pressure to provide for access by all persons suggests that mobile device interfaces need to account for brand differences, cultural differences, differences in computing experience, elderly users, and users with disabilities (e.g., visual, aural, mobility). The effects of poor usability may mean that users cannot complete their tasks or will not be satisfied with the results. This suggests the importance of user-centered design activities in each of the usability areas (user interface, external accessory interface, and service interface). Accessibility is an important design issue and must be considered when user-centered design is applied.

In trying to meet stakeholder usability expectations, MobileApp developers should attempt to answer these questions to assess the out-of-the-box readiness of the device:

- Is the user interface consistent across applications?
- Is the device interoperable with different network services?
- Is the device acceptable in terms of stakeholder values[9] in the target market area?

Eisenstein [Eis01] claims that the use of abstract, platform-neutral models to describe a user interface greatly facilitates the development of consistent, usable multiplatform user interfaces for mobile devices. Three models in particular are useful. A *platform model* describes the constraints imposed by each platform to be supported. A *presentation model* describes the appearance of the user interface. The *task model* is a structured representation of the tasks a user needs to perform to meet her task goals. In the best case, model-based design (Chapter 9) involves the creation of databases that contain the

9 Brand, ethical preferences, moral preferences, cognitive beliefs.

models and has tool support for generating user interfaces for multiple devices automatically. Utilizing model-based design techniques can also help designers recognize and accommodate the unique contexts and context changes that are present in mobile computing. Without an abstract description of a user interface, the development of mobile user interfaces can be error prone and time consuming.

INFO



Mobile User Interface Design Considerations

Design choices affect performance and should be examined early in the user interface design process. Ivo Weevers [Wee11] posted several mobile user–interface design practices that have proven to be helpful when designing mobile applications:

- **Define user interface brand signatures.** Differentiate the app from its competitors. Make the core signature elements of the brand the most responsive, because users will use them over and over.
- **Focus the portfolio of products.** Target the platforms that are most important to the success of the app and the company. Not all platforms have the same number of users.
- **Identify the core user stories.** Make use of techniques that require stakeholders to prioritize their needs as a way to reduce a lengthy list of requirements and to consider the constrained resources available on mobile devices.
- **Optimize user interface flows and elements.** Users do not like to wait. Identify potential bottlenecks in user work flow and make sure the user is given an indication of progress when delays occur. Make sure that the time to display screen elements is justified in terms of user benefits.
- **Define scaling rules.** Determine the options that will be used when information to be displayed is too large to fit on the screen. Managing functionality, aesthetics, usability, and performance is a continual balancing act.
- **User performance dashboard.** The dashboard is used to communicate the product's current state of completion (e.g., number of use stories implemented), its performance relative to its targets, and perhaps comparisons to its competitors.
- **Champion-dedicated user interface engineering skills.** It is important to understand that the implementation of layout, graphics, and animation has performance implications. Techniques to interleave rendering of display items and program execution can be helpful.

13.2.2 Lessons Learned

de Sá and Carriço [Des08] contend that there are important differences between developing conventional software and developing mobile applications. Software engineers cannot continue to use the same conventional techniques they have used and expect them to be successful. They suggest three approaches for the design of mobile applications:

Usage Scenarios. Described in Chapter 12, usage scenarios must consider context variables (location, user, and device) and transitions between contextual scenarios (e.g., user moves from bedroom to kitchen or switches from stylus to a finger). de Sá and Carriço have identified a set of scenario-variable types that should be considered in developing the user scenarios—locations and settings, movement and posture, devices and usages, workloads and distractions, user preferences.

Ethnographic Observation.[10] This is a widely used method for gathering information about representative users of a software product as it is being designed. It is often difficult to observe users as they change contexts, because the observer must follow users for long periods of time, something that could raise privacy concerns.[11] A complicating factor is that users seem to complete tasks differently in private settings than in social settings. The same users may need to be observed performing tasks in multiple contexts while monitoring transitions, as well as recording user reactions to the changes.

Low-Fidelity Paper Prototypes (e.g., cards or Post-it notes). This is a cost-effective usability assessment approach in user interface design that can be used before any programming takes place. It is important for these prototypes to be similar in size and weight and for their use to be allowed in a variety of contexts. It is also important that the sketches or text displays be true to size and for the final product to be of high quality. Placement and size of user interface widgets (e.g., buttons or scrollbars) must be designed so that they will not disappear when users extend their screens by zooming. The interaction type (e.g., stylus, joy stick, touch screen) needs to be emulated in the low-fidelity prototype (e.g., colored pen or push pin) to check placement and ease of use. Later prototypes may then be created to run on the targeted mobile devices once the layout and placement issues have been resolved.

INFO



MobileApp Design Mistakes

Joh Koester [Koe12] posts several examples of mobile design practices that should be avoided:

- **Kitchen sink.** Avoid adding too many features to the app and too many widgets on the screen. Simple is understandable. Simple is marketable.
- **Inconsistency.** To avoid this, set standards for page navigation, menu use, buttons, tabs, and other user-interface elements. Stick to a uniform look and feel.
- **Overdesigning.** Be ruthless when designing apps. Remove unnecessary elements and wasteful graphics. Do not be tempted to add elements just because you think you should.
- **Lack of speed.** Users do not care about device constraints—they want to view things quickly. Preload what you can. Eliminate what is not needed.
- **Verbiage.** Unnecessarily long, wordy menus and screen displays are indications of a mobile product that has not been tested with users and developers who have not spent enough time understanding the user's task.
- **Nonstandard interaction.** One reason for targeting a platform is to take advantage of the user's experience with the way things are done on that platform. Where standards exist use them. This needs to be balanced with the need to have an application appear and behave the same way on multiple devices when possible.
- **Help-and-FAQ-itis.** Adding online help is not the way to repair a poorly designed user interface. Make sure you have tested your app with your targeted users and repaired the identified defects.

10 *Ethnographic observation* is a means determining the nature of user tasks by watching users in their work environment.

11 Asking users to fill out anonymous questionnaires may have to suffice when direct observation is not possible.

13.3 MOBILE ARCHITECTURES

Services computing[12] and cloud computing[13] enable the rapid development of large-scale distributed applications based on innovative architectural designs [Yau11]. These two computing paradigms have made it easier and more economical to create applications on many different mobile devices (e.g., laptop computers, smartphones, and tablets). These two paradigms allow resource outsourcing and transfer of information technology management to service providers while at the same time mitigating the impact of resource limitations on some mobile devices. A service-oriented architecture provides the architectural style (e.g., REST),[14] standard protocols (e.g., XML[15] and SOAP[16]), and interfaces (e.g., WSDL)[17] needed for mobile development. Cloud computing enables convenient, on-demand network access to a shared pool of configurable computing resources (servers, storage, applications, and services).

Service computing allows mobile developers to avoid the need to integrate service source code into the client running on a mobile device. Instead, the service runs out of the provider's server and is loosely coupled with the applications that make use of it through messaging protocols. A service typically provides an application programming interface (API) to allow it to be treated like an abstract black box.

Cloud computing lets the client (either a user or program) request computing capabilities as needed, across network boundaries anywhere or any time. The cloud architecture has three layers, each of which can be called as a service (Figure 13.1). The *software as service* layer consists of software components and applications hosted by third-party service providers. The *platform as service* layer provides a collaborative development platform to assist with design, implementation, and testing by geographically distributed team members. *Infrastructure as a service* provides virtual computing resources (storage, processing power, network connectivity) on the cloud.

Mobile devices can access cloud services from any location at any time. The risks of identity theft and service hijacking require providers of mobile services and cloud computing to employ rigorous security engineering techniques (Chapter 18) to protect their users.

Taivalsaari [Tai12] points out that making use of cloud storage can allow any mobile device or software features to be updated easily on millions of devices worldwide. In fact, it is possible to virtualize the entire mobile user experience so that all applications are downloaded from the cloud.

12 *Services computing* focuses on architectural design and enables application development through service discovery and composition.

13 *Cloud computing* focuses on the effective delivery of services to users through flexible and scalable resource virtualization and loading balancing.

14 *Representation State Transfer* describes a networked Web architectural style where the resource representation (e.g., a Web page) places the client in a new state. The client changes or transfers state with each resource representation.

15 *Extensible Markup Language* (XML) is designed to store and transport data, while HTML is designed to display data.

16 *Simple Object Access Protocol* is a specification for exchanging structured information in the implementation of Web services in computer networks.

17 *Web Services Description Language* is an XML-based language for describing Web services and how to access them.